Data Mining

Block 3 - Trees, ensembles & honest evaluation

Daniel Wlazło

SGH Warsaw School of Economics - SMMD-ADA

20 October 2026

Today, hour by hour

1. **Recap** - the baseline we must now beat ~5 min
2. **Decision trees** - models made of rules ~30 min
3. **SVM interlude** - margins and the kernel trick ~10 min
4. **Ensembles** - forests and boosting ~25 min
5. **Honest evaluation** - CV, ROC/AUC, calibration, cost ~45 min
6. **Leakage** - the trap finally springs ~15 min
7. **Explaining models** - importance & SHAP ~15 min
8. **Lab** - notebook `03_trees_ensembles_evaluation` ~35 min

Breaks roughly every 50 minutes.

Last week in five sentences

1. Judge models only on **unseen data** - train/test is the minimum honest protocol.
2. Our **logistic baseline** reads risk through coefficients and odds ratios - and Block 1's indicator is its strongest feature.
3. **Regularisation** trades a little bias for a lot of variance; every model family has a leash.
4. Accuracy lies under imbalance - read the **confusion matrix** and price its cells.
5. The **threshold** is a business decision. (Today it finally gets its price tag.)

Baseline on the table: test AUC 0.731 on 30,000 real clients. Complexity, earn your keep.

Two promises for today

Models that bend

- decision trees: models made of rules
- forests: many trees voting
- gradient boosting: trees correcting each other

Numbers you can trust

- cross-validation: a distribution, not a lucky draw
- ROC/AUC, PR, calibration: three different questions
- expected cost: metrics meet money

And one demolition: the `collections_contact` trap, armed since Block 1, goes off today.

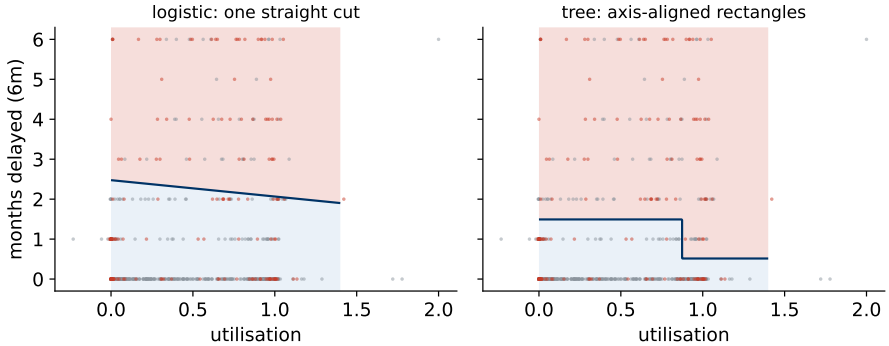
From a line to rules

Logistic regression draws one straight boundary. A decision tree asks nested yes/no questions:

```
if pay_delay_recent >= 2:
    if utilisation > 0.8:  risk = HIGH
    else:  risk = MEDIUM
else:
    risk = LOW
```

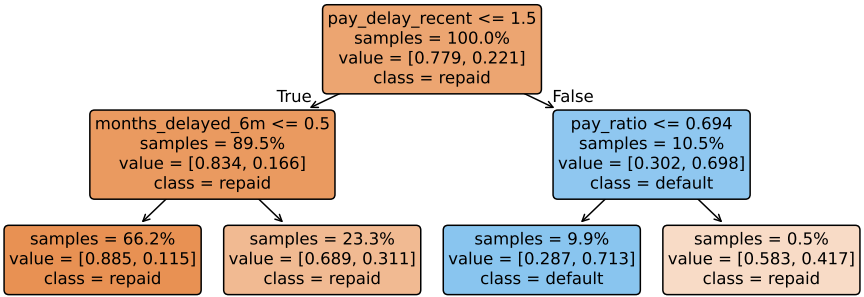
- Each question splits the data; each leaf holds a prediction (here: the leaf's default rate).
- It reads like a credit policy - which is exactly why business people trust trees instantly.
- And note: the second question only applies *inside* the delayed branch - trees model interactions natively (Block 2's product columns, for free).

The boundary picture



- Same two features, same data: logistic draws one straight cut; the tree tiles the space with axis-aligned rectangles.
- More rectangles = more flexibility = a longer walk down the U-curve. The leash question is coming.

Anatomy of a fitted tree (ours, depth 2)



- Predicting = walking one path from root to leaf; the leaf's class mix is the probability estimate. Reading a path aloud gives an explanation for free.
- Look at the root: the tree opens with `pay_delay_recent` at the two-months-behind cliff - agreeing with the logistic coefficients that payment history is where the signal lives. Two very different algorithms, same verdict.

How a split is chosen

At every node, try every feature and every cut-point; keep the split that makes the children purest:

$$\text{Gini}(\textit{node}) = 1 - p_0^2 - p_1^2 \qquad \Delta = \text{Gini}_{\text{parent}} - \tfrac{n_L}{n}\,\text{Gini}_L - \tfrac{n_R}{n}\,\text{Gini}_R$$

- p_k = class shares in the node. Pure leaf: Gini $= 0$; 50/50 node: Gini $= 0.5$. (Entropy, $-\sum_k p_k \log_2 p_k$, is the same idea with logs - rankings of splits rarely differ.)
- Growth is greedy: best split *now*, no lookahead - fast, and not guaranteed optimal.
- Repeat recursively until a stopping rule (the leash) says stop.

Gini by hand (exam-friendly)

Parent node: 1,000 customers, 30% default.

$$\text{Gini}_{\text{parent}} = 1 - 0.7^2 - 0.3^2 = 0.42$$

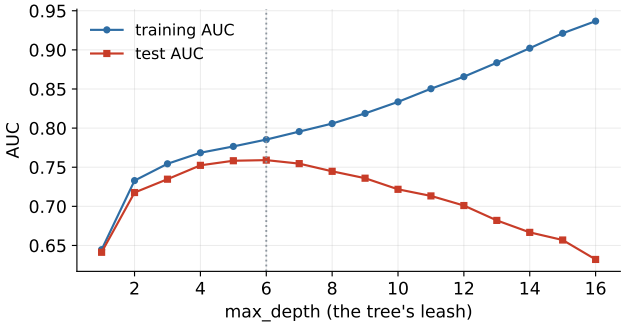
Candidate split on `utilisation` > 0.6:

Child	n	default rate	Gini
left (low util.)	600	10%	$1 - 0.9^2 - 0.1^2 = 0.18$
right (high util.)	400	60%	$1 - 0.4^2 - 0.6^2 = 0.48$

$$\text{weighted} = 0.6 \times 0.18 + 0.4 \times 0.48 = 0.30 \qquad \Delta = 0.42 - 0.30 = 0.12$$

The algorithm computes this Δ for every feature $\times$ every cut-point - and picks the maximum. That's the whole magic.

Where is the tree's leash?



- Unleashed, a tree memorises: training AUC $\to 1$ while test AUC falls - the U-curve, tree edition. Best here: depth $\approx$6 (30,000 rows feed a deeper tree than Block 1's 4,000 ever could); beyond that, memorisation.
- Leashes: `max_depth`, `min_samples_leaf` (no leaf smaller than...), pruning. Same logic as α last week.

A single tree's pathologies

- **High variance.** Nudge the training data and the first split flips - and everything below it changes. One tree is a nervous model.
- **Steps, not slopes.** Within a leaf the prediction is constant: risk that plainly grows with utilisation becomes a staircase.
- **Extrapolation, worse edition.** Beyond the training range a tree predicts the *edge leaf's* value - flat forever. (Block 2's warning, now with a flatline.)

Fix the staircase with depth; fix the nerves with many trees - next section.

Regression trees exist too

- Same machinery, numeric target: leaves predict the mean of their training rows; splits minimise variance (MSE) instead of Gini.
- Same pathologies (staircase, variance) - and the same leash.
- Why care today: gradient boosting fits regression trees to errors even for classification - you need this piece to understand GBDT in ten minutes.

Credit uses them directly, too: LGD and exposure models are regression problems (Block 2's map).

What trees *don't* need

- **No scaling.** Splits compare against cut-points; units are irrelevant. (The scaler stays in the pipeline for the logistic baseline only.)
- **No monotone transforms.** log(income) and income give identical trees - splits are order-based.
- **Ordinal encodings suffice.** Trees can cut an ordered code anywhere; the dummy trap is a linear-model disease.
- **Missing values**: plain sklearn trees still want imputation, but `HistGradientBoosting` handles NaN natively - it learns which side missing values should go. Block 1's structural NaN can stay a NaN.

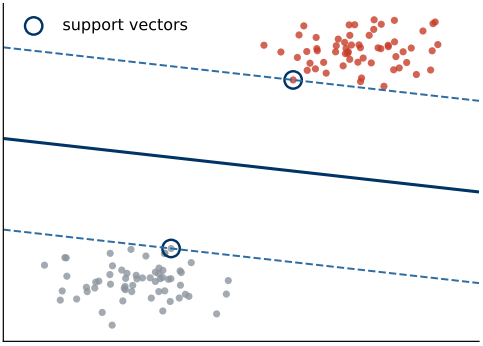
Less preprocessing $\neq$ no thinking: the time-travel test and the missingness taxonomy still apply in full.

Tree vs logistic, side by side

	Logistic regression	Decision tree
Boundary	one straight cut	axis-aligned rectangles
Interactions	engineered by hand	found automatically
Scaling / transforms	required (penalties)	irrelevant
Missing values	impute + indicator	(Hist)GBDT: native
Output	calibrated-ish probabilities	leaf frequencies, chunky
Stability	high	low - nervous
Explanation	coefficients, odds ratios	the path itself

Neither dominates. The interesting question is what happens when trees stop working alone.

Interlude: the widest street (support vector machines)



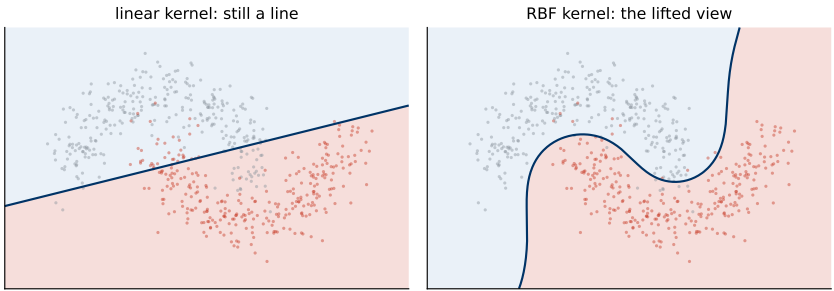
Among all separating lines, pick the one with the widest margin:

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 \ \text{ s.t. } \ y_i(w \cdot x_i + b) \geq 1$$

(margin width $= 2/\|w\|$; soft version adds $C \sum_i \xi_i$ for violations)

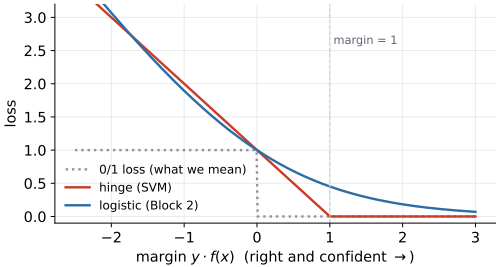
- The line is held up only by the support vectors - the circled border cases; everyone else could vanish.
- Wide street = built-in caution about new customers near the boundary.

The kernel trick: bending without leaving linear algebra



- Idea: map points into a higher-dimensional space where a *line* suffices - computed implicitly through a kernel $K(x, x') = \varphi(x) \cdot \varphi(x')$, never building φ at all.
- The workhorse RBF kernel, $K(x, x') = \exp(-\gamma \|x - x'\|^2)$: similarity decays with distance - and γ is (say it with me) the leash.
- Third route to nonlinearity this course: engineered features (B2), tree splits (today), kernels.

SVM in one loss curve - and in practice



- Hinge loss $\max(0, 1 - yf(x))$: zero once you are right *with margin* - vs Block 2's logistic loss, which never fully forgives. Same recipe as everything else: loss + leash, minimised.
- Practice: scale first (distances again); no native probabilities (calibrate before pricing); training scales poorly past $\sim 10^5$ rows - one reason GBDT took its tabular crown.
- On our PD task (trained on a 6,000-row subsample - the n^2 cost is real): test AUC 0.715, behind the logistic baseline (0.731). Kernels buy bend, but not as cheaply as trees buy the delay cliff - next section.

One tree is nervous - so ask a crowd

- Average many over-flexible, *differently-wrong* models and the individual errors partially cancel: variance drops, bias stays.
- The catch: trees grown on the same data make the *same* mistakes - averaging clones buys nothing.
- Ensembling = manufacturing disagreement between good models, then averaging it away.

Two ways to manufacture disagreement

Bagging: give each tree different *data*. **Boosting**: give each tree a different *job*.

Bagging: bootstrap + aggregate

- **Bootstrap**: sample *n* rows *with replacement* - each tree trains on its own resampled portfolio (∼63% unique rows).
- Grow each tree deep (low bias, high variance), then **aggregate**: average the probabilities.
- Free lunch included: each tree never saw ∼37% of rows - its out-of-bag (OOB) rows give validation-like feedback without a split.

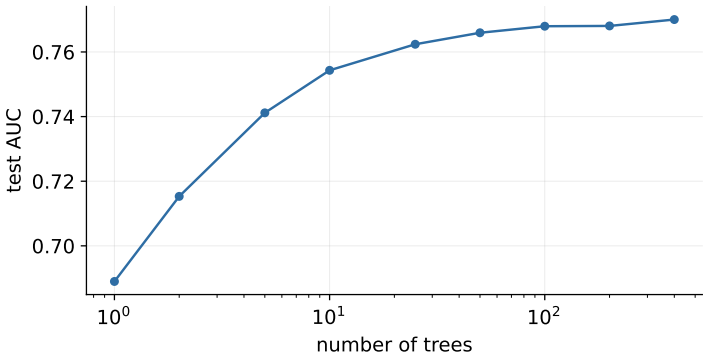
Variance falls like an average of noisy measurements - as long as the trees genuinely disagree.

Random forest = bagging + a feature lottery

- Problem: with one dominant feature (say, utilisation), every bootstrap tree still opens with the *same split* - correlated trees, weak averaging.
- Fix: at every node, allow only a random subset of features (`max_features`) to compete.
- Forced variety ⇒ decorrelated trees ⇒ averaging actually works. That one trick is the "random" in random forest.

Practical defaults: hundreds of deep trees, `max_features` $\approx \sqrt{p}$, `min_samples_leaf` as the leash.

How many trees?



- Test AUC climbs, then plateaus - more trees never overfit; they only cost compute.
- So `n_estimators` is not really a tuning knob: set it "large enough" and tune the *tree-level* leashes instead.

Boosting: a different philosophy

- Bagging builds trees *in parallel* and averages equals. Boosting builds them in sequence: each new tree fits the *errors* of the ensemble so far.
- "Errors" precisely: the gradient of the loss (log-loss for us) - hence *gradient* boosting. Each round is a small regression tree on those gradients (told you they'd matter).
- Predictions accumulate: $F_m(x) = F_{m-1}(x) + \eta \cdot \text{tree}_m(x)$, with learning rate η keeping steps humble.

Bagging attacks variance; boosting attacks bias - and can therefore overfit, so its leash matters more.

GBDT's knobs - and a cautionary tale

- **Learning rate** $\times$ **rounds**: small steps, more of them - the classic robust recipe.
- **Shallow trees** (depth 2–4 / few leaves): each round adds a weak, humble learner.
- **Early stopping**: watch a validation slice, stop when it stops improving - GBDT's leash.

Leashes are sized to the data

On this 21,000-row training set, sklearn's default GBDT already scores **0.774** - defaults are tuned for data this size. The same defaults on Block 1's little synthetic portfolio scored 0.651 against 0.714 tuned: small data needs short leashes. Check, never assume - in either direction.

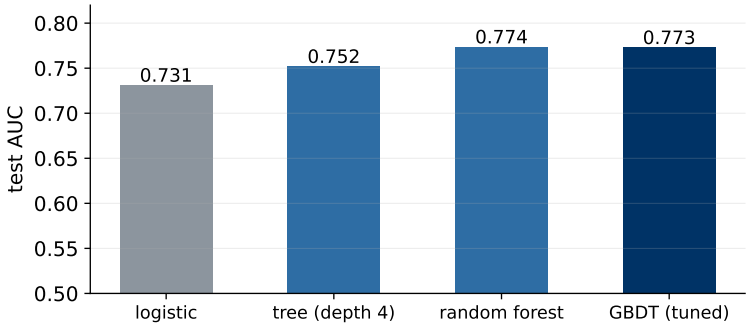
Industry names for the same idea: XGBoost, LightGBM, CatBoost.

RF vs GBDT

	Random forest	GBDT
Trees built	in parallel, independent	in sequence, corrective
Attacks	variance	bias (then overfits)
Tree size	deep	shallow
Key leash	leaf size, feature lottery	rate, rounds, early stop
Tuning effort	low - hard to ruin	moderate - easy to ruin
Typical tabular result	strong	state of the art

Rule of thumb: RF as the robust first ensemble; GBDT when you are ready to tune - and to validate properly.

The tournament on our portfolio



- Every rung of the ladder pays: logistic 0.731 → single tree 0.752 → forest 0.774 / tuned GBDT 0.773.
- On real behavioural data, complexity earns its keep - about +4 AUC points from baseline to ensemble. Is that gap real? The CV section will certify it.

Why complexity wins *here* - and didn't in Block 1's world

- The real portfolio is full of what linear models must be hand-fed: the repayment-status cliff (one missed month is news; the second is a siren), saturating utilisation, delay $\times$ limit interactions. Trees carve these natively.
- Block 1's synthetic world was additive in log-odds by construction - logistic regression's home turf, and complexity politely declined there. Same course, both outcomes, both honest.
- The meta-lesson outlives both datasets.

You cannot know in advance

Which family wins is an *empirical* question, decided per dataset, against a baseline, on honest numbers. Never by fashion.

Tuning without tears

- **Start from defaults**, change one leash at a time; keep a log (Block 1's evidence discipline).
- **Random search beats grid search** for the same budget - important knobs get more distinct values.
- **Tune on cross-validation, confirm on test** - the three-samples contract from Block 2, now load-bearing.
- **Stop early**: tuning yields little once the leash is roughly right; feature work usually pays more (Block 1, always Block 1).

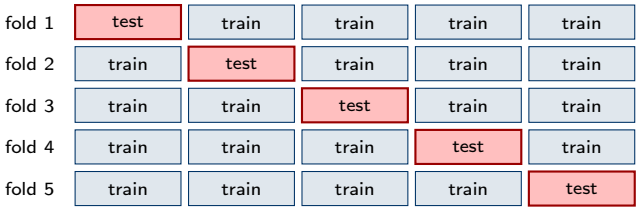
sklearn: `RandomizedSearchCV` - CV built in, pipeline-friendly.

Why one split can lie

- Our test set is 9,000 customers - one draw from the space of possible test sets. A lucky draw flatters; an unlucky one slanders.
- At this size the noise is smaller, but differences of a few tenths of a point of AUC can still be *sampling noise*, not model quality - and most real projects have far fewer rows.
- We compared four models on that single draw last section - how much should we trust the ranking?

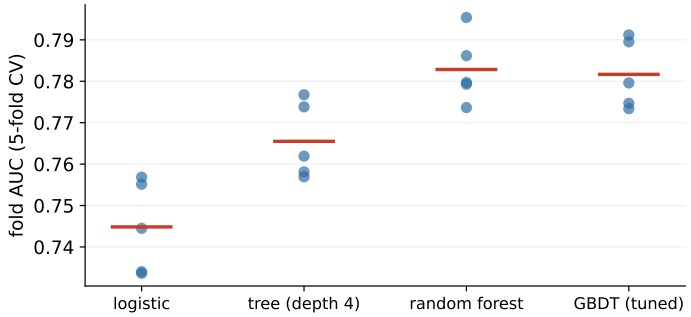
We need a distribution of scores, not a point. Enter cross-validation.

k-fold cross-validation



- Split the *training* data into k folds; train on $k-1$, score on the held-out fold; rotate. Every row gets scored exactly once by a model that never saw it.
- Output: k scores $\rightarrow$ mean $\pm$ spread. ($k = 5$ or 10; stratified for classification.)

CV on our portfolio



- Logistic 0.745 ± 0.010, tree 0.766 ± 0.008, forest 0.783 ± 0.007: at 21,000 training rows the spreads are tight - and the tree-vs-logistic gap ($\sim$0.02) clears its spread several times over. Certified, not lucky.
- Report mean $\pm$ sd, always. A model "winning" by less than the spread hasn't won anything yet - and now you can tell.

CV hygiene

1. **Stratify** the folds - each fold keeps the 22% default rate.
2. **The pipeline goes *inside*** - scaler, imputer, encoder refit on each fold's training part. Fitting transforms once on all data leaks across every fold boundary. This is why pipelines exist.
3. **CV chooses, the test set confirms** - hyperparameters and model family are picked on CV; the untouched test set delivers one final number.
4. **Never** cross-validate on data that includes your test set.

$k = 5$: cheaper, noisier. $k = 10$: slower, tighter. Repeated CV when the decision is close.

Time makes splits harder

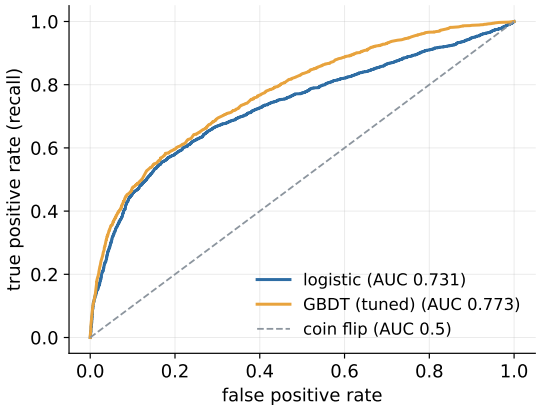
- Credit portfolios drift: products change, policies change (selection bias, Block 1!), the economy cycles.
- Random CV mixes 2023 and 2025 customers in both directions - the model "sees the future" and scores optimistically.
- The credit standard: **out-of-time validation** - train on the past, test on a later window; CV *within* the training period for tuning.
- Our synthetic portfolio has no dates - your project data or your first job will. Ask "does time matter here?" before every split.

Beyond one threshold: judge the ranking

- Last week's precision and recall judged the model *at one threshold* - one confusion matrix out of a continuum.
- But the threshold belongs to the business and may move. What we should judge first is the model's ordering of customers by risk.
- Idea: sweep the threshold from 1 to 0, trace what happens to the error rates - every threshold becomes one point of a curve.

That curve is the ROC; its area is the ranking's quality.

The ROC curve



- x: false positive rate (good customers flagged); y: true positive rate (defaulters caught).
- Each point = one threshold; the diagonal = coin flip.
- Choosing an operating point on this curve *is* choosing a threshold - the business conversation, drawn.

AUC: one number for the ranking

The probabilistic meaning (memorise this one)

AUC = the probability that a randomly chosen *defaulter* is scored riskier than a randomly chosen *good customer*.

- Anchors: 0.5 = dice; 1.0 = crystal ball. Ours: logistic 0.731, tuned GBDT 0.773.
- Threshold-free and insensitive to class imbalance - judges the *ranking*, nothing else.
- What it does not tell you: whether probabilities are honest (calibration) or what a decision costs (thresholding). Three questions, three tools.

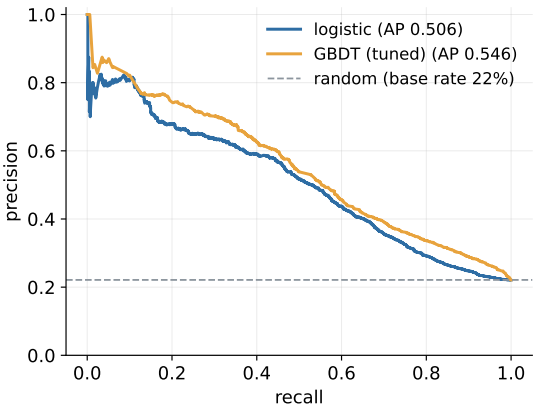
The credit dialect: Gini and KS

- Credit scoring rarely says "AUC". It says Gini:

$$\text{Gini} = 2 \cdot \text{AUC} - 1 \qquad \text{(ours: logistic 0.46, GBDT } 2 \times 0.773 - 1 \approx 0.55\text{)}$$

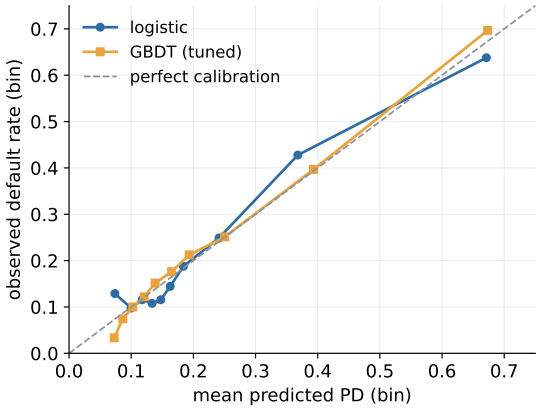
- Same information, rescaled: dice $= 0$, crystal ball $= 1$. A retail PD model with Gini 0.4–0.6 is normal furniture.
- You will also meet **KS**: the maximum gap between the score distributions of goods and bads - one more ranking statistic, same family.
- Interview survival tip: when a risk manager says Gini, they mean this - not the Gini impurity from the tree section. Same Corrado Gini, different formula.

Precision–recall: when positives are rare



- Random guessing floors at the base rate (here 22%) - PR curves punish false alarms among rare positives.
- At 1–5% default rates (or fraud's 0.1%), ROC can look flattering while precision is dismal - check both.
- One number: average precision (AP).

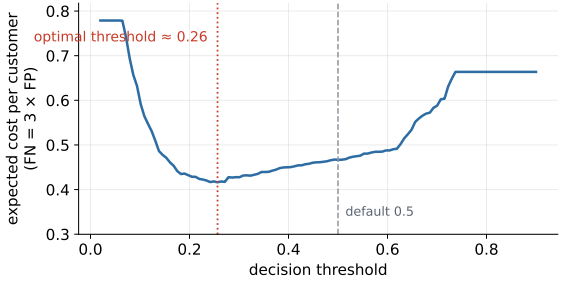
Calibration: is a predicted 0.3 really a 0.3?



- Bin customers by predicted PD; compare with the bin's *observed* default rate. Diagonal = honest.
- Ranking pays for approval order; calibration pays for pricing and provisions - interest rates are set on the probability itself.
- Repairs exist: Platt scaling, isotonic regression (`CalibratedClassifierCV`). And recall Block 2: `class_weight` breaks calibration by design.

From metrics to money: expected cost

$$\mathrm{E}[\text{cost}](t) = \frac{C_{FN} \cdot FN(t) + C_{FP} \cdot FP(t)}{n} \qquad t^* = \arg\min_t \mathrm{E}[\text{cost}](t)$$



- Price the cells (here $C_{FN} = 3 \times C_{FP}$; real low-default books push 10–50×), sweep the threshold, minimise.
- Optimum ≈ 0.26, not 0.5 - and the move cuts expected cost by ∼11%. Same model, better decision. Block 2's cliffhanger: resolved.

The PD evaluation stack

#	Question	Tool	Frequency
1	Does it rank risk?	AUC / Gini, CV mean $\pm$ sd	every model
2	Are probabilities honest?	calibration curve, recalibrate	before pricing
3	Where do we cut?	expected cost vs threshold	with the business
4	Does it stay good?	out-of-time tests, monitoring	forever

In that order. A perfectly calibrated model that ranks badly prices garbage accurately; a great ranker with dishonest probabilities misprices everything.

Metric cheat sheet

Metric	Question	Threshold-bound?	Classic trap
Accuracy	share correct	yes	lies under imbalance
Precision	flagged → right?	yes	ignores missed positives
Recall (TPR)	positives caught?	yes	100% by flagging everyone
F1	precision+recall blend	yes	blind to costs and TNs
AUC / Gini	ranking quality	no	says nothing about calibration
Average precision	ranking, rare positives	no	less intuitive to explain
Calibration error	honest probabilities?	no	great ranking can hide it
Expected cost	money at the cut-off	yes	needs costs you must ask for

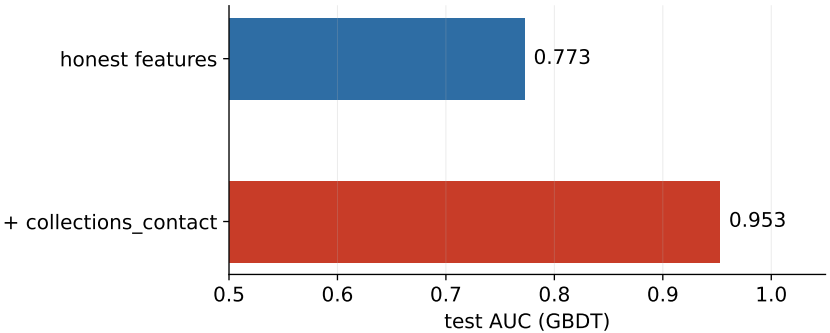
One rule survives every situation: never a single metric, never on training data - now with a toolbox behind it.

The suspect has been in the room since Block 1

- `collections_contact` - "did collections call this customer?" - shipped with the portfolio on day one, flagged do not use.
- Time-travel test: collections calls happen mostly *after* default. At application time this value does not exist.
- Today we do what you should never do in production - on purpose, with instruments attached.

Hypothesis: the model will look magnificent.

The detonation



- One added column: AUC 0.773 → 0.953. The best model this course will ever produce - and completely worthless: in production the feature is unknowable at decision time.
- Nothing crashed. No warning fired. The only alarms were *yours*: the time-travel test, and "too good to be true".

Why leakage is so seductive

- The leaky feature looks like your best feature - importance rankings crown it instantly.
- Every honest check passes: CV loves it, the test set loves it - because the leak sits on *both* sides of every split.
- The only failing dataset is the one you don't have yet: production.
- Which is why leakage is caught by *reasoning about features*, not by metrics: metrics are its accomplices.

A leakage bestiary

Species	Example
Post-outcome field	`collections_contact`, `account_closed_reason`
Target twin	`months_overdue` predicting default
Transform leak	scaler / imputer / encoder fitted on all data
Duplicate leak	same customer lands in train *and* test
Group leak	multiple loans of one person split across sides
Time-travel join	feature table joined with post-decision snapshot

The last three are the sneaky ones: no single column looks guilty - the *procedure* leaks.

The smell test & the checklist

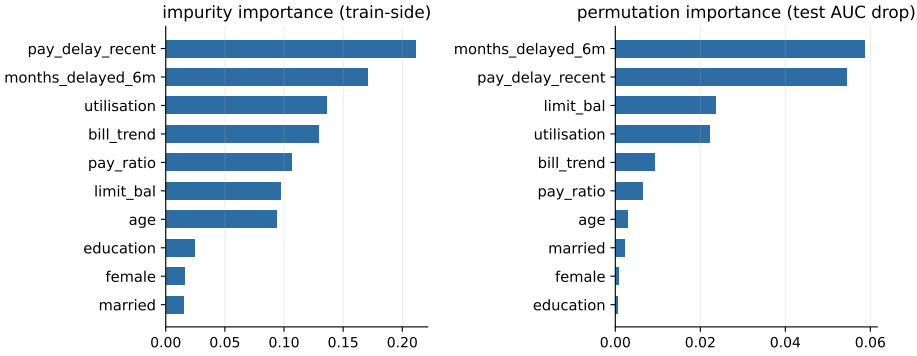
Smells (any one ⇒ stop and audit)

- AUC far above the domain's normal range (retail PD: Gini 0.4–0.6)
- one feature dominating importance
- test score ≫ CV score, or CV suspiciously tight

Checklist (before believing any good result)

- every feature has an answer to "when is this value known?" - written down
- all transforms live inside the pipeline; keys checked for duplicates; groups split together
- time in the data ⇒ out-of-time test exists

Which features matter? Two answers



- **Impurity importance** (left): how much Gini each feature removed - free, train-side, and biased (loves high-cardinality features; splits credit among correlated ones).
- **Permutation importance** (right): shuffle one column on the *test set*, measure the AUC drop - slower, model-agnostic, honest about what the model *uses*.

SHAP in one slide

- Remember Block 2's receipt - each feature's exact contribution to one customer's log-odds? For logistic regression that decomposition is free.
- SHAP generalises the receipt to any model: for one prediction, attribute the deviation from the average outcome across features, fairly (Shapley values, from game theory).
- Local: one customer's bars (adverse-action material). Global: average |contribution| = an importance ranking with direction.
- Costs and caveats: computation; correlated features share credit ambiguously; and it explains the model, not the world.

Lab: permutation importance today; curious? `pip install shap` and point it at your tournament winner at home.

Domain knowledge as a constraint: monotonicity

- Credit intuition: risk should not *decrease* as utilisation rises. But an unconstrained tree ensemble can produce local dips - artefacts of noise.
- Modern GBDT accepts monotonic constraints: force the score to be non-decreasing in chosen features (`monotonic_cst` in sklearn).
- Three birds, one stone: regularisation (fewer wiggles), explainability (no "more debt, less risk" surprises), and regulator comfort.
- The same move as Block 1's binning and Block 2's coefficients: *domain knowledge entering the model on purpose*.

Explanations explain the model - not the world

- Importance $\neq$ causality: utilisation "mattering" means the *model* leans on it, not that reducing it lowers risk.
- Correlated features trade places freely between runs - remember the lasso picking one twin (Block 2).
- Legitimate uses: debugging (a dominant weird feature = leakage smell), monitoring (importance drift), communication (with the caveats spoken aloud).
- Policy conclusions need causal designs - beyond this course, but not beyond your career.

Laboratory

Notebook `03_trees_ensembles_evaluation` - laptops out.

Lab today

Notebook `03_trees_ensembles_evaluation`

- cross-validate the Block 2 baseline
- grow a tree, read it, leash it
- run the tournament: tree vs RF vs GBDT
- ROC, PR, calibration on the winner
- detonate the leak yourselves
- expected cost: pick the threshold

Done =

- you can say *which model won, by how much, and whether the gap beats the CV spread*
- you can name *your* threshold and defend it with a cost curve

Optional extras

- feed raw NaNs to HistGBDT
- monotonic constraints

Summary

What to take home from Block 3.

Block 3 in five sentences

1. Trees turn data into **rules**, find interactions natively, and are too nervous to work alone.
2. Ensembles manufacture disagreement - **bagging** calms variance, **boosting** chips at bias - and every leash is still the U-curve.
3. Judge models by **CV mean** $\pm$ **spread**; judge rankings by **AUC/Gini**, rare positives by **PR**, probabilities by **calibration**, decisions by **expected cost**.
4. **Leakage** makes models look brilliant precisely because every metric is its accomplice - only feature reasoning and out-of-time tests catch it.
5. Complexity must **earn its keep** against a baseline - and on the real portfolio it pays at every rung, about +4 AUC points end to end.

Project status & reminder - topics locked today

- Topic descriptions were due at 17:10 today. Teams and topics are now final - from here on, you build.
- You now own both required model families: a linear baseline and a tree ensemble. The rubric's evaluation criterion (20%) reads: *"correct, leakage-aware"* - today defined both words.
- Minimum evaluation kit for the project: CV mean $\pm$ sd, ROC/AUC *and* PR, a calibration look, one cost-based threshold argument, and a written time-travel audit of your features.

That kit, executed cleanly, is most of the difference between a passing project and a memorable one.

Exam practice - multiple choice

Q1. A risk report states "Gini = 0.55". This refers to:

a. the Gini impurity used to choose tree splits
b. the ranking metric $2 \cdot \text{AUC} - 1$
c. the share of defaults in the portfolio
d. the depth of the decision tree

Q2. Model A: CV AUC 0.745 ± 0.010. Model B: 0.752 ± 0.011, same folds. The honest conclusion:

a. B wins - deploy it
b. the gap is within the fold spread - not yet evidence of a real difference
c. A wins - it has the smaller spread
d. cross-validation cannot compare models

Exam practice - open question

In the style of the term-0 exam (∼60% open questions)

A colleague adds one feature to the PD model and test AUC jumps from 0.77 to 0.95. They propose shipping it. What do you suspect, which two checks do you run, and what makes this failure mode dangerous?

4-6 sentences. "Too good to be true" is the start of the answer, not the end.

Next week: clustering & dimensionality reduction.

No labels. New questions.

Keep learning - Block 3 reading

- **Breiman** - *Random Forests* (Machine Learning, 2001) - the original forest paper: bagging, OOB and importance from the source.
- **Fawcett** - *An Introduction to ROC Analysis* (Pattern Recognition Letters, 2006) - the standard ROC/AUC paper, very readable.
- **Kaufman, Rosset & Perlich** - *Leakage in Data Mining* (KDD 2011) - a bestiary of real leaks; today's detonation, industrialised.
- **scikit-learn User Guide** - *Model evaluation* section: every metric we used, with the fine print.